

Claude Certified Architect — Foundations 考前突击手 册

对照官方 Exam Guide 全部 29 个 Task Statement, 零遗漏覆盖 适用人群: 想快速
通过考试的实战派

考试基本信息

- 5 个 Domain 及权重: D1 Agentic Architecture (27%) | D2 Tool Design & MCP (18%) | D3 Claude Code Config (20%) | D4 Prompt Engineering (20%) | D5 Context & Reliability (15%)
- 题型: 全选择题, 单选, 猜错不扣分 → 不会就猜, 别空着
- 及格线: 720/1000
- 6 个场景随机抽 4 个: 客服 Agent、Claude Code 代码生成、多 Agent 研究系统、开发者工具、CI/CD 集成、结构化数据提取

30 秒速记口诀: 架构 27 提示 20 代码 20 工具 18 可靠 15 (前两大 Domain 占 47%, 拿下就稳了)

Domain 1: Agentic Architecture & Orchestration (27%)

7 个 *Task Statement* | 占比最大，必须全拿

1.1 Agentic Loop (自主循环机制)

什么是 **Agentic Loop**：让 Claude 自己循环工作直到任务完成。

```

你发请求给 Claude
↓
Claude 返回响应
↓
检查 stop_reason
↓
"tool_use" ? → 执行工具 → 结果传回 Claude → 继续循环
"end_turn" ? → 任务完成 → 回复给用户
    
```

stop_reason 字段：API 返回的结构化信号，专门控制循环 -

"tool_use" → Claude 想用工具 → 继续 - **"end_turn"** → Claude 说完了
→ 结束

三大反模式（必考！看到就排除）： - **✗** 不要用自然语言解析判断循环终止（如检测 "Is there anything else?"） - **✗** 不要靠最大迭代数作为主要停止机制 - **✗** 不要检查 assistant text 内容作为完成标志

速记： **stop_reason** 是唯一正确的循环控制信号

1.2 Coordinator-Subagent 模式

Hub-and-spoke 架构：coordinator 是中央枢纽，管所有 subagent 通信、错误处理、信息路由。子 agent 之间不直接通信。

Coordinator 职责：任务分解 → 委派 → 结果聚合 → 决定下一步

三个关键规则： 1. coordinator 必须明确划分研究空间，给每个 subagent 分配不重叠的范围（按子话题或数据源类型分） 2. coordinator 应动态选择调用哪些 subagent，而不是每次都走完整流水线 3. coordinator 可以搞迭代精炼循环：评估合成结果 → 发现缺口 → 重新派子 agent 补充 → 直到覆盖充分

Subagent 上下文：必须显式传递，不自动继承父上下文

1.3 Subagent 调用、上下文传递与并行生成（官方新增重点）

Task 工具：Agent SDK 中用来生成子 agent 的机制 - coordinator 的 **allowedTools** 必须包含 **"Task"** 才能调子 agent - 子 agent 不会自动继承父上下文，也不会在多次调用间共享记忆

AgentDefinition 配置：定义每种子 agent 的 description、system prompt、工具限制

并行生成子 agent：coordinator 在一轮响应中发出多个 **Task** 调用 → 并行执行，不要一个一个分多轮发

上下文传递最佳实践： - 把前序 agent 的完整发现直接放进子 agent prompt（如搜索结果 + 文档分析 → 传给合成 agent） - 用结构化格式分离内容和元数据（source URL、文档名、页码），保留引用归属 - coordinator prompt 应指定研究目标和质量标准，而非逐步操作指令 → 让子 agent 自适应

fork_session：从共享分析基线创建独立探索分支（如对比两种测试策略）

速记： **Task** 工具生子 agent | **allowedTools** 必含 **Task** | 一轮多 **Task** = 并行

1.4 多步工作流与强制执行 / 升级交接

Prompt vs 程序级保障（核心判断）：

后果	方案	可靠性
格式不一致、回复质量波动	Prompt 优化	概率性（够用）
退错款、认错人、安全漏洞	程序级硬限制	确定性（必须）

速记： `prompt` 是"建议"，程序是"法律"

程序级前置条件示例： `process_refund` 被阻塞，直到 `get_customer` 返回已验证的 `customer ID` → 没验证就根本调不了退款

多问题请求拆解：客户一条消息提多个问题 → 拆成独立项 → 用共享上下文并行调查 → 合成统一回复

结构化交接摘要（升级给人工时）： 包含： `customer ID`、问题根因、退款金额、建议操作 → 人工接手后不需要读整段对话

1.5 Agent SDK Hooks（确定性拦截）

Hook = 在特定事件发生时自动执行的代码，确定性的，Claude 控制不了。

HOOK 类型	触发时机	用途	示例
PreToolUse	工具执行之前	权限检查、参数验证、阻止高风险操作	退款 > \$500 → 拦截 → 转人工
PostToolUse	工具执行之后	结果转换、格式统一	Unix 时间戳 → ISO 8601 人类可读

Hook vs Tool 的区别（必考！）： - Hook = 自动触发（确定性，Claude 都不知道） - Tool = Claude 主动选择调用（概率性，可能选错或忘了用）

什么时候选 **Hook** 而非 **Prompt**: - 业务规则需要100% 执行 → Hook - 格式偏好、风格要求 → Prompt 就够

速记: **Hook = 自动 + 确定 | Tool = 选择 + 概率 | 钱和安全 → 必须 Hook**

1.6 任务分解策略

模式	适用场景	示例
Prompt Chaining (固定流水线)	可预测的多步骤审查	per-file 分析 → cross-file 整合
Dynamic Decomposition (动态分解)	开放式探索任务	先扫描结构 → 发现重点 → 动态生成子任务

Multi-pass Review (高频考点): - 大 PR 单 pass → 注意力稀释 → 拆成 per-file local pass + cross-file integration pass - local pass 找局部 bug, integration pass 找跨文件数据流问题

自适应调查计划: 先 mapping 结构, 识别高影响区域, 然后生成优先级化计划, 随着依赖发现动态调整

1.7 会话状态、恢复与分叉

机制	用途	适用场景
<code>--resume <session-name></code>	恢复命名会话	跨工作时段继续调查
<code>fork_session</code>	从共享基线创建独立分支	对比两种重构方案
新会话 + 注入摘要	从头开始但带上上下文	旧工具结果已过期

选择决策: - 之前的上下文大部分还有效 → `--resume` - 之前的工具结果已过期 (代码改了) → 新会话 + 结构化摘要更可靠 - 恢复后有文件改动 → 告诉 agent 具体改了哪些文件, 做定向重分析, 而非全部重来

Domain 2: Tool Design & MCP Integration (18%)

5 个 Task Statement

2.1 工具描述是 LLM 选工具的首要依据

好的工具描述应该包含：- 这个工具做什么 - 接受什么输入格式 - 什么时候用它（示例查询） - 什么时候不用它（和相似工具的边界）

工具名称/描述重叠 → 模型分不清：如 `analyze_content` vs

`analyze_document` → 直接改名消除重叠（如改为 `extract_web_results`）

拆分泛化工具：一个 `analyze_document` 做太多事 → 拆成

`extract_data_points` + `summarize_content` + `verify_claim_against_source`

System prompt 影响工具选择：关键词敏感指令可能无意中引导 agent 选错工具 → 工具描述没问题但选择有系统性偏差时 → 查 **system prompt**

2.2 工具设计原则

消除无效状态：一个工具接受太多参数组合且很多无效 → 拆分为多个工具 -

例： `log_workout` （23% 参数搭配错误） → 拆成 `log_cardio_workout` + `log_strength_workout`

竞态条件：两步操作之间有时间窗口 → 合并为原子操作 - 例：查可用时段 + 预约 → `find_and_book_appointment`

统一返回 **schema**：多来源数据格式不同 → 工具内部做格式转换，返回统一 **schema**

最小权限原则：工具太通用 → **agent** 会滥用 → 换成功能更窄的工具 - 例：

fetch_url（啥都能访问）→ **load_document**（只能加载文档格式）

工具数量控制：**agent** 有 18 个工具 → 选择可靠性下降 → 缩减到 4-5 个角色相关工具

2.3 MCP 结构化错误响应（高频考点）

工具返回错误时要分类，不能统一返回 “Operation failed”：

字段	作用	示例
isError	告诉 agent 调用失败	true
errorCategory	错误类型	transient / validation / business / permission
isRetryable	该不该重试	true （超时） / false （业务规则违规）
人类可读描述	给用户看的信息	"Refund exceeds policy limit"

关键区分（必考！）： - **isRetryable: false** + 业务规则违规 → **agent** 不要无意义重试，直接告知用户 - “0 results”（查询成功，没匹配） ≠ “timeout”（查询没完成） → 性质不同，**coordinator** 决策不同

2.4 **tool_choice** 配置

值	行为	适用场景
"auto"	Claude 自己决定用不用工具（可能不调）	一般对话
"any"	必须调工具，但不指定哪个	需要保证调工具（多 schema 不知类型时）
{"type": "tool", "name": "xxx"}	强制调指定工具	需要确定性输出（先提取再处理）

Scoped Tool (有限工具授权)：子 agent 频繁需要验证简单事实 → 给它一个受限的 `verify_fact` 小工具 (85% 简单查询自己解决)，复杂验证仍走 coordinator

2.5 MCP (Model Context Protocol)

MCP 核心价值 = 标准化接口，一次构建，处处复用 - 写一个 MCP server → 所有 MCP 兼容的 AI 应用都能直接连 - MCP 不提供：自动重试、自动鉴权/限流、更快的协议

MCP 配置文件：

文件	作用
项目级 <code>.mcp.json</code>	团队共享的 MCP 服务器配置
用户级 <code>~/.claude.json</code>	个人的 MCP 服务器配置

密钥管理：永远不把 secret 写死在配置文件里，用 `${ENV_VAR}` 环境变量展开

MCP Resources (官方新增考点)：- MCP 可以暴露内容目录 (issue 摘要、文档层级、数据库 schema) - 让 agent 知道有什么数据可用，不需要通过试探性工具调用去发现

选社区 vs 自建：标准集成 (如 Jira) 优先用社区 MCP server，团队专属 workflow 才自建

MCP 工具描述要详细：描述写太简单 → agent 会优先选内置工具 (如 Grep) 而不用更强的 MCP 工具

2.6 内置工具选择指南（官方新增考点）

工具	用途	典型场景
Grep	搜索文件内容	找函数调用者、定位错误信息、搜 import
Glob	按文件名/路径匹配	找 <code>**/*.test.tsx</code>
Read	读取完整文件	理解文件内容、跟踪 import 链
Write	创建/完整重写文件	Edit 找不到唯一锚点时的备选
Edit	精确替换文件中的唯一文本	小范围修改
Bash	执行系统命令	运行测试、安装依赖

Edit 失败处理：文本不唯一 → 用 **Read** 读完整文件 + **Write** 重写（不要死磕 Edit）

代码探索策略：Grep 找入口 → Read 跟 import → 逐步理解（而非一次性读所有文件）

跟踪函数使用：先找所有 `export` 的名字 → 再逐个 **Grep** 搜索调用位置

Domain 3: Claude Code Configuration & Workflows

(20%)

6 个 Task Statement

3.1 CLAUDE.md 配置层级

层级	位置	生效范围
用户级	~/.claude/CLAUDE.md	只对该用户生效，不共享
项目级	.claude/CLAUDE.md 或 根目录 CLAUDE.md	跟着 repo 走，团队共享
目录级	子目录 CLAUDE.md	只对该子目录生效

诊断思路：当”部分人有效、部分人无效”时 → 规则放错了层级（该项目级却放了用户级）

@import 语法（官方新增考点）： - 引用外部文件保持 CLAUDE.md 模块化（如每个 package 导入各自的规范文件） - 示例：在子包的 CLAUDE.md 中 @import 相关的 coding standards 文件

拆分大 CLAUDE.md：太长的 CLAUDE.md → 拆到 .claude/rules/ 下的专题文件（如 testing.md、api-conventions.md、deployment.md）

/memory 命令：用于验证加载了哪些 memory 文件，诊断跨会话行为不一致

3.2 Skill vs Path Rules vs CLAUDE.md 三选一（高频考点）

机制	触发方式	适用场景
<code>.claude/rules/</code> (path rules)	文件路径匹配 (glob pattern) , 自动、确定性	按文件类型/位置应用不同规范
<code>.claude/skills/</code> (skills)	任务触发 (手动调用或 Claude 选 择加载)	特定任务的工作流
<code>CLAUDE.md</code>	始终加载, 不区分文件	全局通用规则

关键区分：看到”自动” + “按文件类型/路径” → path rules

3.3 Path Rules 的 frontmatter（官方新增细节）

```
---
paths: ["terraform/**/*"]
---
Terraform 文件编辑规范...
```

- `paths` 字段用 glob pattern → 只有编辑匹配文件时规则才加载
- 减少无关上下文、节省 token
- 选 path rules 还是子目录 CLAUDE.md? → 当规范要应用到散布在多个目录的文件时（如所有 `**/*.test.tsx` ），用 path rules；当规范只属于一个目录时，用子目录 CLAUDE.md

3.4 Frontmatter 配置对比

Rules 和 Skills 都有 frontmatter，字段不同：

RULES 的 FRONTMATTER	SKILLS 的 FRONTMATTER
字 <code>paths</code> （管哪些文件）、 <code>description</code> 段	<code>context: fork</code> 、 <code>allowed-tools</code> 、 <code>argument-hint</code>

Skills 的 frontmatter 详解：

字段	作用	示例
<code>context: fork</code>	在隔离子 agent 中运行，不污染主会话	代码分析、头脑风暴
<code>allowed-tools</code>	限制 skill 可用的工具	只允许读操作，禁止写/删
<code>argument-hint</code>	没传参数时提示用户输入	"请输入要分析的文件路径"

3.5 Commands vs Skills vs 配置位置

COMMANDS		SKILLS
frontmatter	不支持	支持 <code>context: fork</code> 、 <code>allowed-tools</code> 、 <code>argument-hint</code>
能力	纯文本指令	可隔离运行、限制工具、提示参数

Skills = Commands + 超能力（frontmatter 配置）

配置文件位置总览：

要共享什么	项目级（团队共享）	用户级（个人）
指令/规则	<code>.claude/CLAUDE.md</code>	<code>~/.claude/CLAUDE.md</code>
Skills	<code>.claude/skills/</code>	<code>~/.claude/skills/</code>
Commands	<code>.claude/commands/</code>	<code>~/.claude/commands/</code>
MCP 服务器	<code>.mcp.json</code>	<code>~/.claude.json</code>
Path rules	<code>.claude/rules/</code>	—

万能规律：项目目录下的 = 团队共享（版本控制），`~/` 下的 = 个人专属

个人 Skill 定制：在 `~/.claude/skills/` 下用不同名字创建个人变体，不影响团队

3.6 Plan Mode vs Direct Execution

	PLAN MODE	DIRECT EXECUTION
适用	大规模/跨文件架构变更、需求模糊、多种方案	简单、明确、小范围改动
示例	微服务重构、45+ 文件的库迁移	单文件 bug fix、加一个日期校验
好处	先探索设计再动手，避免返工	快速完成

Explore subagent: verbose 探索阶段用 Explore subagent 隔离输出 → 主会话只收精简摘要 → 防止 context window 爆炸

组合用法: Plan mode 做调研 → 确定方案后切 Direct execution 执行

3.7 迭代精炼技术（官方新增考点）

三种迭代模式:

模式	适用场景	做法
具体 I/O 示例	自然语言描述 → 模型理解不一致	给 2-3 个 input/output 对
测试驱动迭代	复杂实现	先写测试 → 分享失败结果 → 逐步修正
Interview Pattern	不熟悉的领域	让 Claude 先提问（缓存失效策略？故障模式？） → 再实现

批量 vs 逐个修 bug: - 多个 bug 互相影响 → 一条消息说清全部问题 - 多个 bug 相互独立 → 逐个迭代修复

3.8 CI/CD 集成

三个 CLI flag（递进关系）:

FLAG	作用
<code>-p</code> / <code>--print</code>	非交互模式，CI 必加，不加就挂起
<code>--output-format json</code>	输出 JSON 格式，程序可解析
<code>--json-schema</code>	强制输出符合指定 schema

CI 中的标准用法

```
claude -p "分析这个PR" --output-format json --json-schema '{"type":"object"...}'
```

CLAUDE.md 给 CI 提供上下文: 在 CLAUDE.md 中写测试标准、fixture 规范、review 标准 → CI 调用的 Claude 自动获得这些上下文

会话隔离：生成代码的 Claude session 审查自己 → 确认偏差 → 用独立实例审查

避免重复评论：re-run review 时，把之前的 findings 带上，指示 Claude 只报告新的或未解决的问题

提供已有测试文件：让 Claude 知道哪些场景已覆盖 → 不会建议重复的测试用例

Domain 4: Prompt Engineering & Structured Output (20%)

6 个 *Task Statement*

4.1 Prompt 优化的层级递进（必考！考了 5+ 次）

第一层：Prompt 模糊（没有判断标准）
→ 精确化标准 (explicit criteria)

第二层：标准清楚但执行不一致
→ Few-shot examples (给具体示例示范)

第三层：动态变化的遗漏（每次缺的不一样）
→ 自我审查 (Evaluator-Optimizer pattern)

关键词快速判断： - 题目说“no clear criteria” / prompt 模糊 → 选精确化标准 - 题目说“instructions already added but inconsistent” → 选 **few-shot** - 题目说“gaps vary by case” → 选自我审查

False Positive 管理： - 高误报 + 信任崩塌 → 先关掉高误报类别（止血），保留高精度类别，改好了再启用 - “be conservative” / “only report high-confidence” 这种模糊指令没用 → 要用具体分类标准

4.2 Few-shot Examples 设计原则

1. 针对出错场景——不要教模型已经会的东西
2. 展示推理过程——不只给答案，给“为什么选这个”
3. 数量精而不多——2-4 个精准示例 > 10-15 个泛泛示例

Few-shot 能做的事： - 模糊场景下的工具选择（展示推理过程） - 区分可接受代码模式 vs 真正的问题（减少误报） - 不同文档结构的正确提取方式（内联引用 vs 参考文献） - 减少提取任务中的幻觉（如非正式度量单位的处理）

Few-shot 不是万能药（速记：先排除再上 few-shot）： - 基础标准没有 → 先精确化，不是给例子 - 工具描述/名字有问题 → 先修描述/改名 - 涉及钱/安全 → 程序级保障，不靠例子 - 动态遗漏 → 自我审查 - system prompt 有无意的路由指令 → 先查 prompt

4.3 结构化输出（JSON Schema + Tool Use）

最可靠方式：定义一个带 JSON schema 的 tool，用 `tool_use` 强制结构化输出

Tool use 消除的是语法错误，不是语义错误！ -  消除：JSON 格式错误、字段缺失 -  不消除：行项目求和不等于 total、值填到错误字段

`tool_choice` 与结构化输出：

场景	用什么
多个提取 schema，不知道文档类型	<code>tool_choice: "any"</code>
必须先提取元数据，再做后续处理	<code>tool_choice: {"type": "tool", "name": "extract_metadata"}</code>
一般对话	<code>tool_choice: "auto"</code>

Schema 设计要点： - 字段值不在源文档中 → 设为 **nullable**（不是 required）→ 防止幻觉 - 情感判断不了（讽刺）→ 加 `"unclear"` enum 选项 - 需要扩展性 → `"other"` + detail string 字段 - 格式不统一的源数据 → prompt 里加格式标准化规则

自动交叉校验：加 `calculated_total`（行项目求和）+ `stated_total`（文档上写的），不匹配时标记人工审查

4.4 验证、重试与反馈循环

验证失败重试：直接重试 = 重复同样的错误 → 把原始文档 + 失败的提取结果 + 具体验证错误都带上再试

重试的局限：信息根本不在提供的文档中 → 重试再多次也没用 → 识别何时重试无效

`detected_pattern` 字段（官方新增考点）：- 在结构化 finding 中加入 `detected_pattern` 字段 - 记录是什么代码模式触发了这个 finding - 开发者 dismiss 某条 finding 时 → 可以系统性分析误报模式

自校验流程：提取 `calculated_total` + `stated_total` → 不匹配时加 `conflict_detected: true`

4.5 Batch API

	SYNCHRONOUS API	BATCH API
延迟	实时响应	最长 24 小时
成本	全价	50% 折扣
适用	阻塞工作流（pre-merge）	非阻塞、延迟可容忍（夜间分析）

三大限制：1. 不能等的工作流不能用 2. 需要多轮工具调用的工作流不能用（fire-and-forget，不支持中途交互） 3. 没有延迟 SLA（最长 24h，不是保证快）

`custom_id` 字段（官方新增考点）：每个 batch request 带 `custom_id` → 用于关联请求和响应 → 失败时只重交失败的（通过 `custom_id` 识别）

批次调度计算：文档持续到达 + 24h 处理窗口 + 30h 内出结果 → 每 6 小时提交一批

大批量处理策略：先用小样本优化 prompt → 全量用 Batch API 提交 → 失败的逐批重试

4.6 Multi-instance 与 Multi-pass Review (必考!)

自我审查局限：同 session 生成 + 审查 → 模型保留推理上下文 → 确认偏差
→ 不太会质疑自己的决定

解法：用独立 **Claude** 实例审查（没有生成时的推理上下文）

Multi-pass Review： - 大 PR 单 pass → 注意力稀释 + 前后矛盾 - 拆成：
per-file local pass（找局部问题）+ cross-file integration pass（找跨文件数据流问题）

置信度自报：让模型对每个 finding 自报置信度 → 实现校准化的 review 路由

关键词敏感偏差：工具描述没问题但选择有系统性偏差（某关键词出现时 78% 选错）→ 查 system prompt 中有没有无意的路由指令

Domain 5: Context Management & Reliability (15%)

5 个 Task Statement

5.1 上下文窗口管理

Lost in the Middle: LLM 对长输入的开头和结尾关注度高，中间容易漏 - 解法：关键发现摘要放在最前面 + 加 **section headers** 帮助导航中间内容

长对话摘要丢失关键细节：摘要天生是有损的 - 解法：关键事实（金额、日期、订单号、政策引用）提取到持久化的 “**case facts**” 块，不参与摘要，每轮注入 prompt

上游输出过大：子 agent 输出总量 155K tokens，下游 agent 最优 50K - 解法：让上游 agent 只返回结构化关键信息（事实 + 引用 + 相关性评分），从源头减少 token - 不要加中间摘要 agent（治标不治本，应从源头减）

5.2 升级决策 (Escalation)

什么时候该升级给人工： - 政策空白 → 没有规则可遵循，不能自己编 - 需要主观判断 → 超出 agent 能力范围（需要同理心判断） - 不可逆的高风险操作 → 需要人工确认 - 超出 **agent** 权限 → 如退款金额超过 agent 授权上限

什么时候不该升级： - 有标准流程可循的（物流争议有处理流程） - 多个问题同时来（agent 能处理多问题） - 客户可能改主意（猜测不是升级理由）

多匹配结果消歧义：工具返回多个匹配 → 别猜，问用户提供额外标识（邮箱、电话、订单号）

升级 vs 程序保障的区分： - 升级 = 需要人的判断（没有规则可循） - 程序保障 = 有规则但需要 100% 执行（退款上限、身份验证）

5.3 优雅降级与部分失败处理

核心区分（必考！）： - “0 results”（查询成功，没有匹配）→ 有意义的空结果 - “timeout”（查询没完成）→ 失败，需要决定是否重试

绝对不能做的：默默跳过不上报（silent skip）→ 藏错误 = 最终输出有缺口但没人知道

正确做法： - 部分数据源失败 → 用已有数据继续合成 - 在输出中明确标注覆盖范围（哪些领域有充分数据、哪些有缺口、哪些源失败了） - 数据冲突 → 保留两个值 + 标注冲突 → 让 coordinator 决定，不要自己选

5.4 Agent 间信息流优化

问题：子 agent 输出太多 → 下游合成质量下降 原则：源头过滤优于后处理摘要

方案	效果
✅ 让上游 agent 只返回结构化关键信息	从源头减少
❌ 加中间摘要 agent	多一层处理，治标不治本

结构化传递格式：内容和元数据分离 - 内容：事实、发现、引用 - 元数据：source URL、文档名、页码、相关性评分

5.5 自我审查与质量保证

Evaluator-Optimizer 模式： 1. 生成草稿回复 2. 用 checklist 自检（政策？时间线？下一步？回答了客户问题？） 3. 发现缺口 → 补全 4. 输出最终回复

适用场景：动态变化的遗漏（每次缺的东西不一样）

字段级置信度分数： - 让模型对每个提取字段输出置信度 - 低置信度 → 优先人工审查 - 高置信度也可能有错 → 分层随机抽样（每个置信度分组都抽一部分）

速记： 低置信优先审，高置信也要抽

高频考点决策树速查

"输出不一致"怎么选？

- Prompt 有没有清晰的判断标准？
 - 没有 → 精确化标准
 - 有，但执行不统一 → 题目提到"加了指令但不一致"？
 - 是 → few-shot examples
 - 遗漏是动态变化的？ → 自我审查 (Evaluator-Optimizer)

"工具选错了"怎么选？

- 工具描述/名字是否清晰、无重叠？
 - 不清晰 / 有重叠 → 修描述 / 改名
 - 清晰，但特定关键词触发偏差 → 查 system prompt
 - 清晰，但模糊场景选错 → few-shot (针对模糊场景 + 展示推理)

"要不要升级给人工"怎么选？

- 有没有政策/规则覆盖这个情况？
 - 没有 (政策空白) → 升级
 - 有标准流程 → 不升级, agent 自己处理
- 后果是否涉及金钱/安全？
 - 是 → 程序级保障 (不是升级, 是硬限制)

"Prompt 够还是要程序保障"怎么选？

- 错误后果严重吗？
 - 格式/风格/效率问题 → Prompt 优化够了
 - 金钱/安全/身份验证 → 程序级硬限制 (Hook / 前置条件)

“该用什么配置机制”怎么选？

- 规则是否始终适用？
 - 是 → `CLAUDE.md`
 - 只在特定文件类型/路径时适用 → `path rules (.claude/rules/)`
 - 只在特定任务时需要 → `skills (.claude/skills/)`
- 谁需要这个规则？
 - 团队所有人 → 项目级 (`.claude/`)
 - 只有自己 → 用户级 (`~/`)

“输出太多/上下文爆了”怎么选？

- 是子 agent 输出太多？
 - 是 → 源头过滤 (让子 agent 返回结构化关键信息)
 - 是 skill 输出太 verbose？
 - 是 → `context: fork` 隔离
 - 是代码探索输出太多？
 - 是 → `Explore subagent`
 - 是长对话丢关键信息？
 - 是 → `case facts` 块持久化
-

解决问题的优先级（万能判断框架）

1. 先优化 prompt（写清标准 + 给例子）—— 最简单、最快、成本最低
2. 再调工具/流程（修描述、拆工具、改工作流）
3. 最后上架构/基础设施（额外模型、ML 分类器、程序级保障）

例外：涉及金钱、安全、合规时直接上程序级保障

易混淆概念对比速查

概念 A	概念 B	区别
Hook	Tool	Hook 自动确定性 vs Tool Claude 选择概率性
<code>isRetryable: true</code>	<code>isRetryable: false</code>	超时可重试 vs 业务规则违规不要重试
"0 results"	"timeout"	查询成功无匹配 vs 查询失败没完成
Plan Mode	Direct Execution	复杂/模糊/跨文件 vs 简单/明确/单文件
<code>tool_choice: "any"</code>	<code>tool_choice: "auto"</code>	必须调工具 vs 可能不调
Path Rules	子目录 CLAUDE.md	跨目录按文件类型 vs 单目录全局
Skills	Commands	有 frontmatter 超能力 vs 纯文本
<code>context: fork</code>	Explore subagent	反复使用的隔离 skill vs 临时探索
升级 (escalation)	程序保障 (hook)	需要人判断 vs 需要 100% 执行规则
Sync API	Batch API	实时阻塞 vs 24h 50% 折扣
Few-shot	JSON Schema	教 Claude 怎么做 vs 验证做得对不对
精确化标准	Few-shot	没标准先加标准 vs 有标准但执行不一致
<code>--resume</code>	新会话 + 摘要	上下文大部分有效 vs 旧结果已过期

基础概念速查

术语	解释
Agentic Loop	Claude 自主循环（用工具 → 看结果 → 再用工具...）直到 <code>stop_reason: "end_turn"</code>
stop_reason	API 返回的循环控制信号： <code>"tool_use"</code> 继续 / <code>"end_turn"</code> 结束
Hub-and-spoke	coordinator 中心辐射架构，子 agent 不直接通信
Task tool	Agent SDK 中生成子 agent 的工具
AgentDefinition	子 agent 的配置（description、system prompt、工具限制）
fork_session	从共享基线创建独立探索分支
Inline（内联）	信息直接展示在行内，不用额外操作去查看
Atomic Operation	一步完成，中间不可打断，避免竞态条件
Confirmation Bias	倾向于支持自己已有的判断。同 session 生成 + 审查 = 确认偏差
Graceful Degradation	部分功能失败时用已有数据继续工作，同时标明缺口
Least Privilege	给 agent 的工具/权限只够做它该做的事
Prompt Chaining	固定顺序的多步骤流水线
Dynamic Decomposition	根据中间发现动态生成子任务
MCP Resources	MCP 暴露的内容目录，让 agent 知道有什么数据可用
custom_id	Batch API 中关联请求和响应的标识符
detected_pattern	结构化 finding 中记录触发代码模式的字段
@import	CLAUDE.md 中引用外部文件的语法